Experiment No 3

Aim: Machine Translation Using Transformer Models.

Theory:

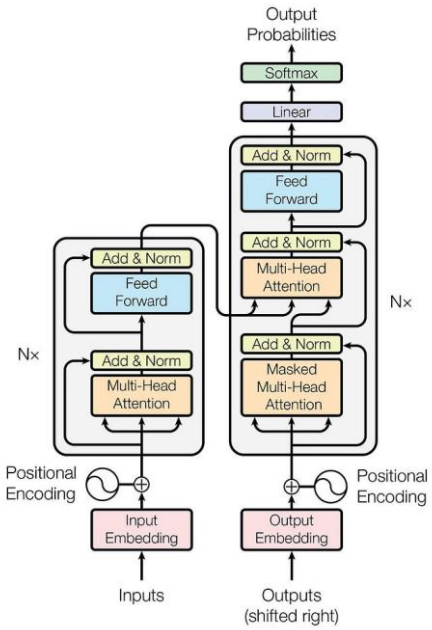
Introduction

The Transformer model, introduced in the paper "Attention is All You Need" (Vaswani et al., 2017), revolutionized the field of natural language processing (NLP) by eliminating the need for recurrent neural networks (RNNs) and convolutional networks for sequence modelling tasks. Instead, the Transformer relies entirely on a mechanism called self-attention to process and relate elements in a sequence.

Key Components of the Transformer:

1. **Self-Attention Mechanism:**
   - The self-attention mechanism allows the model to focus on different parts of an input sequence to compute a representation of the sequence. It captures dependencies between tokens, regardless of their position in the sequence.
   - Given an input sentence, self-attention computes the relevance (attention scores) of each word with respect to every other word in the sentence.
   - For example, in the sentence "The cat sat on the mat," self-attention allows the model to understand that "the" relates to both "cat" and "mat," despite being separated by other words.
2. **Multi-Head Attention:**
   - To capture different relationships between words, the Transformer uses **multi-head attention**, where the self-attention mechanism is applied multiple times in parallel, and the outputs are concatenated. This enables the model to focus on different aspects of the sequence simultaneously.
3. **Positional Encoding:**
   - Since the Transformer does not have a built-in notion of word order (unlike RNNs), it uses **positional encodings** to inject information about the position of words in the sequence. These encodings are added to the input embedding, ensuring that the model can differentiate between words based on their position in a sentence.
4. **Encoder-Decoder Structure:**
   - The Transformer consists of an **encoder** and a **decoder**, each composed of several identical layers.
     - **Encoder:** The encoder processes the input sequence by applying self-attention and feed-forward layers. It generates a set of encoded representations.

- **Decoder:** The decoder takes the encoder's output and the target sequence (shifted by one step) as input. It uses both self-attention and **encoder-decoder attention** (which focuses on the encoder's output) to generate the final translated output.



5. **Feed-Forward Networks:**
   - Each attention mechanism is followed by a **position-wise feed-forward network**, which consists of two fully connected layers applied independently to each position in the sequence. This helps in learning non-linear transformations of the input.
6. **Layer Normalization and Residual Connections:**
   - Each sub-layer in the encoder and decoder has a **residual connection** around it, followed by layer normalization. This helps stabilize training and allows the model to learn more effectively.

Here are the key steps to train a transformer model for **Neural Machine Translation (NMT)** from scratch:

1. Data Collection

The first and most crucial step is to gather a **large parallel dataset** consisting of sentence pairs from the source and target languages (e.g., English-German). A parallel dataset contains text in both languages that are aligned on a sentence level.

- **Open datasets**: Common parallel datasets include Europarl, TED talks, or datasets from the WMT (Workshop on Machine Translation).
- **Custom datasets**: You might also need to create your own dataset, especially if you're working with specialized domains or less common languages.

For example:

- **Source sentence (English)**: "I am learning."
- **Target sentence (German)**: "Ich lerne."

2. Data Pre-processing

Once you have the dataset, the next step is to prepare the data for training. This includes:

- **Cleaning the data**: Remove noisy or corrupted sentences, sentence pairs that are not properly aligned, etc.
- **Tokenization**: Split the sentences into smaller subword tokens. The transformer model works at the subword level rather than whole words. For example:
  - "I am learning" might be tokenized into ["I", "am", "learn", "ing"].
  - "Ich lerne" might be tokenized into ["Ich", "lern", "e"].
- **Create vocabulary**: Build a vocabulary for the source and target languages based on the tokens in your dataset. This vocabulary is used to convert tokens into numerical representations (IDs).
- **Padding and truncation**: Sentences will vary in length, so shorter sentences need to be padded to match the length of the longest sentence in a batch. Longer sentences might be truncated to a maximum length.

3. Model Architecture Setup

Now that your data is ready, you need to build the transformer architecture. The main components include:

- **Encoder**: This part of the model processes the source language sentence (e.g., English) and converts it into a sequence of hidden representations.
  - It uses multiple layers of self-attention and feed-forward networks.
- **Decoder**: The decoder generates the translated sentence (e.g., German) from the hidden representations produced by the encoder.
  - It also uses self-attention and attention mechanisms to focus on the relevant parts of the source sentence.

The transformer model typically consists of multiple layers of encoders and decoders (e.g., 6 layers each), and each layer includes:

- **Multi-head self-attention**: To allow the model to focus on different parts of the sentence.
- **Feed-forward networks**: To transform the attention outputs.

You can initialize the transformer model's weights randomly since you're training from scratch.

4. Training the Model

Once the model is set up, the training process begins. This involves multiple steps:

a) Loss Function

- The model is trained to minimize the **cross-entropy loss**, which measures how well the predicted output matches the actual target sentences. The goal is to increase the probability of correct translations and decrease the probability of incorrect ones.

b) Optimization

- You typically use optimizers like **Adam** or **Adafactor** to update the model's weights during training.
- **Learning rate schedules**: The learning rate is gradually increased in the initial stages (warm-up) and then decreased later.

c) Training Strategy

- You input batches of source sentences (e.g., English) and their corresponding target translations (e.g., German).
- For each batch:
    1. The **encoder** processes the source sentence and generates hidden representations.
    2. The **decoder** uses these hidden representations to predict the target sentence, one token at a time.
    3. The model compares its prediction with the actual target sentence, calculates the loss, and updates the weights accordingly.

d) Teacher Forcing

- During training, **teacher forcing** is used, where the correct previous token is given as input to the decoder rather than the decoder's own previous prediction. This helps the model learn more effectively, especially in the early stages of training.

5. Evaluation Metrics

Once the model is trained, it is essential to evaluate its performance.

- **BLEU Score**: The most common metric for evaluating machine translation models. It compares n-grams (consecutive sequences of tokens) in the machine-generated translation with reference translations.
- **Validation**: You should periodically evaluate the model on a validation set (a part of the dataset not used during training) to ensure it is not overfitting.

6. Hyperparameter Tuning

- **Batch size**: Adjust the number of examples processed at a time.
- **Learning rate**: Fine-tune the learning rate schedule for better performance.
- **Number of layers**: You may adjust the number of encoder and decoder layers based on the complexity of the dataset.
- **Dropout**: Use dropout regularization to avoid overfitting.

7. Model Evaluation and Fine-Tuning

After training the model, you need to test it on unseen data (test set) to evaluate how well it generalizes. You can also fine-tune it on more specific data if needed.

8. Deploying the Model

Once the model has been trained and evaluated, you can deploy it for real-world applications:

- **Batch translation**: Translate large volumes of text efficiently.
- **Real-time translation**: Use the model in applications such as live chat translation or multilingual customer service.

Lab Assignment:

Perform machine translation using Transformers on the following dataset.

cfilt/iitb-english-hindi · Datasets at Hugging Face

English to Hindi Machine Translation

This notebook demonstrates how to build and train a sequence-to-sequence model for English-to-Hindi machine translation using the Hugging Face Transformers library.

```
# Install necessary libraries
!pip install -q transformers datasets sentencepiece sacrebleu
accelerate evaluate
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.0/100.8 kB ? eta -:--:--
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 100.8/100.8 kB 7.9 MB/s eta
0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.0/84.1 kB ? eta -:--:--
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 84.1/84.1 kB 9.3 MB/s eta
0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.0/65.9 kB ? eta -:--:--
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 65.9/65.9 kB 7.1 MB/s eta
0:00:00
```

```
import numpy as np
import torch
from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSeq2SeqLM,
    DataCollatorForSeq2Seq,
    Seq2SeqTrainingArguments,
    Seq2SeqTrainer
)
import evaluate
```

1. Load Dataset

We will use the `cfilt/iitb-english-hindi` dataset from Hugging Face Datasets.

```
dataset = load_dataset("cfilt/iitb-english-hindi")
print(dataset)
```

```
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.
```


```
{"model_id":"7831ec71034142f9a06354d0698da4f7","version_major":2,"vers
ion_minor":0}
```

{"model_id":"c3583624517f46aa91939dbb6f785898","version_major":2,"version_minor":0}

{"model_id":"032fea3c25d243ebb0e278b831aaf845","version_major":2,"version_minor":0}

{"model_id":"1dbdda9cfa8147d9b42aab8b6d604667","version_major":2,"version_minor":0}

{"model_id":"92e9b7b258274d95a246d8a1ca2abfba","version_major":2,"version_minor":0}

{"model_id":"eb96f26711ed44f8b22fcc26ac841d3f","version_major":2,"version_minor":0}

{"model_id":"46f55a3b78fc4833afea5828039ece63","version_major":2,"version_minor":0}

{"model_id":"b380f3c94c0e44de927fbb68ca2b9825","version_major":2,"version_minor":0}

{"model_id":"a62c6ba0995a489cab62d674d87867e4","version_major":2,"version_minor":0}

{"model_id":"56bb8f4410a3495f9d91f43ad9248e1a","version_major":2,"version_minor":0}

{"model_id":"ae300c3ef5cb413a867ad2a095ec2e7b","version_major":2,"version_minor":0}

```
DatasetDict({
    train: Dataset({
        features: ['translation'],
        num_rows: 1659083
    })
    validation: Dataset({
        features: ['translation'],
        num_rows: 520
    })
    test: Dataset({
        features: ['translation'],
        num_rows: 2507
    })
})
```

Split and Sample Data

To speed up training for demonstration, we'll sample a smaller subset of the dataset.

```
train_dataset = dataset["train"].shuffle(seed=42).select(range(5000))
validation_dataset = dataset["validation"].select(range(500))
```

```
test_dataset = dataset["test"].select(range(500))

print(f"Train dataset size: {len(train_dataset)}")
print(f"Validation dataset size: {len(validation_dataset)}")
print(f"Test dataset size: {len(test_dataset)}")
```

```
Train dataset size: 5000
Validation dataset size: 500
Test dataset size: 500
```

2. Initialize Tokenizer and Model

We will use the `google/mt5-base` model and its tokenizer, which is a multilingual T5 model suitable for translation tasks.

```
model_name = "google/mt5-base"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
```


```
{"model_id":"5c7998391863408d9aa8c8ba8cca16d5","version_major":2,"vers
ion_minor":0}

{"model_id":"1c72276e7eed431cacb9cd087a28b694","version_major":2,"vers
ion_minor":0}

{"model_id":"b91def3cb91a4fcca8cfd535557587ae","version_major":2,"vers
ion_minor":0}

{"model_id":"511c375cdfb64f3bb894beefd0579856","version_major":2,"vers
ion_minor":0}

{"model_id":"f32ec5d3119141a1aef0ee968c2b6fab","version_major":2,"vers
ion_minor":0}

{"model_id":"a50d2c342040478a86f1470670f26126","version_major":2,"vers
ion_minor":0}

{"model_id":"863169fd4fd347daa84bd64df6578b3b","version_major":2,"vers
ion_minor":0}

{"model_id":"2457351641374bef9f5b9d334ccf0303","version_major":2,"vers
ion_minor":0}

{"model_id":"4474c1449c0e4d7eabad65b4a243adbf","version_major":2,"vers
ion_minor":0}
```


```
[transformers] The tied weights mapping and config for this model
specifies to tie shared.weight to lm_head.weight, but both are present
in the checkpoints with different values, so we will NOT tie them. You
should update the config with `tie_word_embeddings=False` to silence
this warning.
```

{"model_id":"b500ad7df60642eb950d170399b0d553","version_major":2,"version_minor":0}

3. Preprocess Data

We need to tokenize the English and Hindi sentences and prepare them for the model.

```
max_input_length = 64
max_target_length = 64

def preprocess(example):
    source = example["translation"]["en"]
    target = example["translation"]["hi"]

    inputs = tokenizer(
        source,
        max_length=max_input_length,
        truncation=True
    )

    labels = tokenizer(
        text_target=target,
        max_length=max_target_length,
        truncation=True
    )

    inputs["labels"] = labels["input_ids"]

    return inputs

tokenized_train = train_dataset.map(
    preprocess,
    remove_columns=train_dataset.column_names
)

tokenized_validation = validation_dataset.map(
    preprocess,
    remove_columns=validation_dataset.column_names
)

tokenized_test = test_dataset.map(
    preprocess,
    remove_columns=test_dataset.column_names
)
```

{"model_id":"44f9e3e559fe4ad89a479f691cd88723","version_major":2,"version_minor":0}

{"model_id":"4715ab3282154d4e879a888fce388046","version_major":2,"version_minor":0}

{"model_id":"35897323166e4d679722e74c3496c0a3","version_major":2,"version_minor":0}

4. Data Collator

`DataCollatorForSeq2Seq` will handle padding for batches.

```
data_collator = DataCollatorForSeq2Seq(
    tokenizer=tokenizer,
    model=model
)
```

5. Metrics Definition

We will use BLEU score to evaluate the translation quality.

```
bleu = evaluate.load("sacrebleu")

def compute_metrics(eval_pred):
    predictions, labels = eval_pred

    # Handle potential NaN or out-of-range values in predictions
    # Replace NaNs with pad_token_id, then convert to int64 for safe
decoding
    if np.isnan(predictions).any():
        predictions = np.nan_to_num(predictions,
nan=tokenizer.pad_token_id)

    # Ensure predictions are integer type. Using int64 for robustness.
    predictions = predictions.astype(np.int64)
    # Replace any negative values (like -100 if present) with
pad_token_id
    predictions = np.where(predictions < 0, tokenizer.pad_token_id,
predictions)

    decoded_preds = tokenizer.batch_decode(
        predictions,
        skip_special_tokens=True
    )

    labels = np.where(labels != -100, labels, tokenizer.pad_token_id)

    decoded_labels = tokenizer.batch_decode(
        labels,
        skip_special_tokens=True
    )

    decoded_labels = [[label] for label in decoded_labels] # sacrebleu
```

```
expects list of references

    result = bleu.compute(
        predictions=decoded_preds,
        references=decoded_labels
    )

    return {"BLEU": result["score"]}
```

{"model_id":"73b15a719fcc4b8fa32805ceb3be513b","version_major":2,"version_minor":0}

6. Training Arguments and Trainer

Define the training arguments and instantiate the `Seq2SeqTrainer`.

```
training_args = Seq2SeqTrainingArguments(
    output_dir="./translation_model",
    eval_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    weight_decay=0.01,
    save_total_limit=2,
    num_train_epochs=2,
    predict_with_generate=True,
    logging_steps=100,
    fp16=False # Set to False to avoid numerical instability issues encountered previously
)

trainer = Seq2SeqTrainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_train,
    eval_dataset=tokenized_validation,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)
trainer.train()
```

```
<IPython.core.display.HTML object>
```

{"model_id":"c163406db0824b5a8922f87598fff105","version_major":2,"version_minor":0}

```
/usr/local/lib/python3.12/dist-packages/transformers/generation/utils.py:1676: UserWarning: Using the model-agnostic default `max_length` (=21) to control the generation length. We recommend setting `max_new_tokens` to control the maximum length of the
```

```
generation.
  warnings.warn(

{"model_id":"dc93e7f714ff45d6b8d45888fc40de19","version_major":2,"vers
ion_minor":0}

{"model_id":"2d3f4eef955e42dfb77093a42c20102c","version_major":2,"vers
ion_minor":0}

TrainOutput(global_step=1250, training_loss=7.066655554199219,
metrics={'train_runtime': 1435.0048, 'train_samples_per_second':
6.969, 'train_steps_per_second': 0.871, 'total_flos':
1124368039821312.0, 'train_loss': 7.066655554199219, 'epoch': 2.0})
```

7. Evaluate the Model

Evaluate the trained model on the validation set.

```
results = trainer.evaluate()
print(results)

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

{'eval_loss': 3.0871057510375977, 'eval_BLEU': 0.17572185943735477}
```

8. Translate Function

Define a function to translate an English sentence to Hindi.

```
# Get the device of the model (assuming it's on CUDA if available)
model_device = model.device

def translate(sentence):
    inputs = tokenizer(
        sentence,
        return_tensors="pt"
    ).to(model_device) # Move inputs to the model's device

    outputs = model.generate(
        **inputs,
        max_length=max_target_length # Use the predefined
max_target_length
    )

    return tokenizer.decode(
        outputs[0],
```

```
        skip_special_tokens=True
    )
```

Test Translation

```
sentence = "Artificial Intelligence is changing the world."

print("English:")
print(sentence)

print()

print("Hindi:")
print(translate(sentence))
```

```
English:
Artificial Intelligence is changing the world.

Hindi:
<extra_id_0> और नई चीजों के लिए प्रयोग करने के लिए प्रयोग करने के लिए प्रयोग करने के लिए प्रयोग करने के लिए प्रयोग करने के लिए प्रयोग करने के लिए प्रयोग करने के लिए
```

9. Save Model and Tokenizer

Save the fine-tuned model and tokenizer for future use.

```
model.save_pretrained("./english_hindi_transformer")
tokenizer.save_pretrained("./english_hindi_transformer")
print("Model and tokenizer saved to ./english_hindi_transformer")
```

```
{"model_id":"1179e6b1dd8c496181affffc4d4500fa","version_major":2,"vers
ion_minor":0}
```

```
Model and tokenizer saved to ./english_hindi_transformer
```